UNIVERSITY OF PISA

DEPARTMENT OF INFORMATION ENGINEERING

Project Report for the course of Symbolic and Evolutionary Artificial Intelligence

Photonic Neural Networks for Proactive Blockage Prediction of 6G Connections using Quantization-Aware Training

Project Group:

Valentina Bertei

Alex Sgammato

Francesco Tarchi

ACADEMIC YEAR 2025/2026

Abstract

Ensuring seamless reliability in next-generation high-frequency wireless communication systems, such as millimeter-wave and sub-terahertz networks, is increasingly dependent on proactive blockage prediction. Although deep learning models have proven highly effective for this task, their implementation on conventional hardware faces growing challenges regarding energy consumption and latency. Photonic neuromorphic computing emerges as a promising alternative, offering massive energy savings and ultra-low latency through analog operations. However, deploying neural networks on optical hardware requires adherence to strict physical constraints, including limited numerical precision and mandatory positive data representations. This project explores the feasibility of implementing proactive blockage prediction using Photonic-Aware Neural Networks (PANNs). By integrating Quantization-Aware Training (QAT) and robust architectural design, we enable the model to adapt to these hardware-enforced limitations while maintaining high predictive capabilities. Our experimental evaluation on Scenarios 17-22 of the DeepSense 6G dataset shows that a photonic-constrained architecture matches the unconstrained baseline (86.5-87.4% mean test accuracy) within 0.2%, and that 16-bit and 8-bit QAT incur negligible accuracy loss (86% and 84.2% respectively). At 4-bit and 2-bit precision, standard Z-score normalization causes severe degradation which we show derives from an information bottleneck at the data ingestion stage rather than model capacity. A dedicated quantile-based normalization strategy, designed to maximize entropy across the available hardware states, recovers vast part of this loss, restoring 76.5-78% accuracy at 4-bit and 60-72% at 2-bit precision.

Repository Link

The GitHub repository with all the materials related to the present work can be reached at the following link.

https://github.com/francetarchi/PANNsBlockagePrediction

Contents

1. Introduction

Ensuring seamless reliability and ultra-low latency in next-generation high-frequency communication systems, such as millimeter-wave (mmWave) and sub-terahertz (sub-THz) networks, heavily depends on mitigating *Line of Sight (LOS)* signal disruptions.

Although artificial intelligence, specifically *Deep Learning* applied to early wireless signal fluctuations [5], offers a powerful tool for proactive blockage prediction, scaling these models on conventional hardware is becoming unsustainable. Standard digital systems are severely constrained by the continuous transfer of data between processing and memory units, which results in massive energy consumption and latency [2]. As the computational demands of *state-of-the-art* AI technologies constantly increase, electronic architectures are rapidly approaching their physical limits [2].

Optical computing offers a great alternative to bypass these electronic bottlenecks. By leveraging light to perform analog operations, *Photonic Neuromorphic Computing* delivers minimal latency and massive energy savings. Translating neural network architectures into the optical domain, however, gives rise to the concept of *Photonic-Aware Neural Networks (PANNs)* [2]: models explicitly designed to operate within the strict boundaries of analog hardware. Unlike digital systems, photonic accelerators introduce unique structural challenges, including extremely low numerical precision due to background noise, the necessity for strictly positive data representations, and rigid limitations on convolutional filtering and spatial downsampling [2].

This project investigates the feasibility of blockage prediction using wireless signatures through a PANN architecture. To prevent catastrophic performance drops when transitioning to restricted hardware, the proposed methodology integrates *Quantization-Aware Training (QAT)*, allowing the network to adapt to severe low-precision constraints directly during the learning phase. By applying these hardware-specific numerical and structural limitations to a *Convolutional Neural Network* evaluated on the real-world vehicular sequences of the *DeepSense 6G* dataset [1], this research aims to quantify the trade-off between optical efficiency and predictive accuracy. Ultimately, this work explores how next-generation AI technologies can solve complex telecommunication tasks while strictly adhering to the operational realities of emerging photonic technologies.

2. Literature Review

This project bridges two distinct but highly complementary research domains: proactive link blockage prediction in next-generation wireless networks and the physical implementation of Deep Learning (DL) models on energy-efficient optical hardware. Our methodology is primarily inspired by two foundational works that define the state-of-the-art in these respective fields [5, 2].

2.1 Proactive Blockage Prediction via Wireless Signatures

In the context of millimeter-wave (mmWave) and sub-terahertz (sub-THz) communications, line-of-sight (LOS) link blockages pose significant reliability and latency challenges. To address this, *Wu et al.* proposed a novel approach that relies exclusively on mmWave wireless measurements to proactively predict future dynamic blockages [5]. Their framework leverages "pre-blockage wireless signatures" [5], that are constructive and destructive interference patterns in the received signal power caused by a moving object acting as a scatterer as it approaches the LOS link [5].

In their comprehensive study, the authors investigated four distinct predictive subproblems: predicting the occurrence of a future blockage, estimating the exact timing of the blockage, classifying the blockage type, and determining its moving direction [5]. To solve these tasks, they evaluated different DL architectures, employing *Recurrent Neural Networks (RNNs)* to exploit the temporal dependencies of received signal sequences, and *Convolutional Neural Networks (CNNs)* to extract spatio-temporal features from two-dimensional beam-time heatmaps [5].

Their work successfully demonstrates that both types of networks can analyze these power fluctuations to answer critical predictive questions, including blockage occurrence, timing, type, and moving direction [5]. Particularly relevant to our methodology are the outdoor vehicular environments (*Scenarios 17-22*), where the receiver continuously scans the surroundings using a mmWave phased array codebook of 64 beams [5]. In these settings, CNNs proved to be highly effective due to their capability to extract local features from two-dimensional receive power heatmaps [5].

Our work narrows its focus exclusively to the first subproblem: predicting whether a blockage will occur within a future time window. Since for this binary classification task in outdoor vehicular environments, the CNN architecture developed by *Wu et al.* reached high performances [5], we establish our baseline by mirroring[1] their exact CNN structure and methodology, replicating the specific outdoor scenario trajectories and exploiting the original DeepSense 6G dataset [1], before subsequently attempting to introduce photonic hardware constraints.

2.2 Photonic-Aware Neural Networks

While the CNN models developed for wireless signature detection achieve high predictive accuracy, scaling such modern DL architectures is increasingly bounded by the physical limits of traditional digital electronics. In conventional von Neumann architectures, a primary source of latency and power consumption is the continuous data movement between physically separate computing cells and memory units [2].

In order to overcome these computational bottlenecks, *Photonic Neuromorphic Computing* leverages optical circuits to perform analog computations, improving energy and footprint efficiency by several orders of magnitude [2]. However, deploying neural networks on optical hardware requires abstracting strict physical layer constraints into the model architecture, giving rise to Photonic-Aware Neural Networks (PANNs) [2]. *Paolini et al.* outline several limitations to this analog optical domain [2]:

1. Photonic engines suffer from noise and distortions, restricting their *Effective Number of Bits (ENOB)* to very small resolutions, typically 6 bits or fewer [2].

2. Since information is encoded in the intensity of optical signals, all inputs are strictly required to be positive-valued [2].

3. Third, physical fan-in limitations restrict convolutional kernel sizes to a maximum of 3×3 [2].

To prevent heavy accuracy degradation when shifting to low-precision analog hardware, direct quantization of full-precision weights is avoided [2]. Instead, Quantization-Aware Training (QAT) is employed, utilizing the pseudo-gradient method (*Straight-Through Estimator*) and *DoReFa* quantizers available in libraries such as Larq [2]. This technique simulates low-precision behavior by adding fake quantization nodes during the training forward pass, allowing the network weights to adapt to quantization loss [2].

[1] In the original paper by *Wu et al.* [5], some implementation details about preprocessing steps and training phase are missing or leave room for reader interpretation. Therefore, after having tried to reproduce the exact results of the paper without success, we have made some assumptions about data manipulation and model training that make our work not an exact continuation of *Wu et al.* [5], but rather the study of a partially different prediction problem and its consequent adaptation to PANNs.

Furthermore, architectural adjustments, such as substituting *Max Pooling* layers with *Average Pooling* ones, are necessary to align with optical aggregation capabilities, as the latter is a linear operation easily implemented in photonics [2].

By synthesizing the insights from both works [5, 2], our research adapts a high-performing CNN baseline for wireless signature detection into a strictly constrained PANN framework, ultimately assessing the viability of optical computing for next-generation predictive network management.

3. Methodology

This chapter describes in details the operational pipeline developed to achieve proactive link blockage prediction using wireless signatures under structural and hardware-enforced constraints. The methodology is structured into distinct functional stages.
Our implementation focuses strictly on the first predictive subproblem established by *Wu et al.* [5], isolating the case study only to the CNN architecture[1]: a binary classification task to determine whether a future LOS blockage will occur within a specified look-ahead window.

3.1 Dataset and Scenarios Characterization

The experimental framework of this research is grounded in the DeepSense 6G dataset [1], a large-scale, real-world multi-modal collection specifically engineered to bridge sensing and communication tasks. This study restricts its operational domain to Scenarios 17 through 22, which capture realistic multi-modal outdoor vehicular environments.
In these specific scenarios, the experimental setup comprises a fixed *base station (BS)* transmitting a continuous mmWave signal and a *mobile station (MS)* mounted on a vehicle moving along predefined urban outdoor trajectories. The MS actively scans its environment using a phased array codebook consisting of $N_b = 64$ distinct narrow beams. As the vehicle advances along its trajectory, the surrounding dynamic entities (such as passing buses, trucks, or large structures) act as scatterers and eventual blockers. The dataset records the received signal power across all 64 beam pairs at each time sample t. This continuous stream of multi-beam measurements forms the "wireless signatures" [5] that serve as the primary sensing modality for our predictive model.

3.2 Problem Formulation and Prediction Parameters

To formalize the binary blockage prediction task, we model the sequence of received power profiles as a discrete-time random process. Let $\mathbf{x}_t \in \mathbb{R}^{N_b}$ represent the vector of received signal power levels across the $N_b = 64$ beams at a specific time instant t.
By ideally positioning ourselves at any time instant t, the proactive prediction framework relies on two temporal parameters defined over window frames:

[1]The *Wu et al.* [5] results on the RNN architecture have been completely ignored.

- **Observation window (T_o):** The number of consecutive *past* time samples used as input to the neural network to capture the trend of constructive and destructive signal interference. Basically, this is the information the network examines to make a prediction.
- **Prediction window (T_p):** The number of consecutive *future* time samples used to define how far into the future the network must accurately forecast the LOS link status. This is the information needed to label the correspondent observation window T_o as *blockage* or *non-blockage*.

The parameter T_p represents a fundamental trade-off in network management. A larger T_p provides the network with a broader proactive action window, allowing sufficient time to execute handoff protocols or beam steering procedures before the physical link actually degrades. However, predicting deeper into the future could increase uncertainty, making the task more challenging: as we increase T_p, we are pushing T_o away from the actual blockage time instance, consequently removing pre-blockage signature portions from T_o.
In the DeepSense 6G dataset, the ground-truth link status at any instance t, denoted as $x[t] \in \{0, 1\}$, was manually annotated by the authors using synchronized RGB camera feeds: $x[t] = 1$ indicates a physically *blocked* LOS link, whilst $x[t] = 0$ indicates an *unblocked* LOS link [5].
Following the original formulation, our pipeline evaluates these labels to automatically construct the target binary label y_t for a given observation sequence. The target label is mathematically defined as:

$$y_t = \begin{cases} 0, & \text{if } x[t+\tau] = 0 \quad \forall \tau \in \{1, \ldots, T_p\} \\ 1, & \text{otherwise} \end{cases} \tag{3.1}$$

where $y_t = 1$ denotes an impending link blockage occurring somewhere within the forward look-ahead interval $[t + 1, t + T_p]$, and $y_t = 0$ indicates a stable communication link [5].

3.3 Data Preprocessing and Input Generation

The initial step in our methodology involves managing and processing the raw data from the DeepSense 6G dataset. According to the dataset's architectural guidelines [1], the data for Scenarios 17 to 22 is organized into specific sub-directories. Each scenario folder contains a `resources` directory (holding ground-truth blockage labels), a `unit1` directory (containing camera, mmWave, and GPS data), a `unit2` directory (containing GPS data), and a `scenarioX.csv` file that aligns the visual, wireless, and GPS data for each coherent time step. To optimize processing time and minimize disk space consumption, our extraction script selectively unzips only the strictly necessary files from the original archives. Specifically, for Scenarios 17 through 21, the script extracts only the `scenarioX.csv` file, the `unit1/mmWave_data/` folder, and the corresponding `unit1/label_data/` (or `unit1/label/`) folder, entirely bypassing the heavy GPS and camera image files.

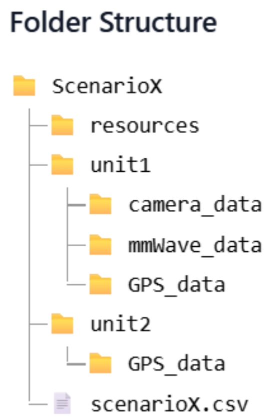


Figure 3.1: Folder structure of the DeepSense 6G scenario datasets.

Notably, Scenario 22 was intentionally excluded: manual verification revealed that data contained in Scenario 22 is an exactly identical duplicate of Scenario 21, but contaminated with extensive missing values (`NaN`). Consequently, excluding this scenario prevents redundant processing and avoids potential bias from incomplete data streams.

Following the extraction, the core preprocessing pipeline generates the structured input datasets for the binary blockage prediction task. The algorithm iterates over ten distinct sizes of the prediction window, specifically $T_p \in \{1, 2, \dots, 10\}$. The pipeline explicitly creates and saves a separate processed dataset for each of these 10 values. Each of these 10 processed datasets are eventually separated into *training*, *validation* and *test* sets. The data processing workflow is structured into three main operational phases: reading, sample identification, and sample creation.

3.3.1 Reading Phase

During this initial stage, the algorithm caches all the ground-truth labels from the text files and the mmWave power data directly into the system's RAM.

3.3.2 Sample Identification Phase

This phase identifies the exact *temporal indices* (time instances t) for both *positive* (blocked) and *negative* (unblocked) samples, using a fixed observation window of $T_o = 16$ time steps. To guarantee that the training, validation and test sets maintain identical dimensions across all ten T_p values, the pipeline evaluates sample validity based on the largest prediction window ($T_p = 10$). Consequently, a sample considered valid for $T_p = 10$ is automatically valid for all shorter prediction windows.

The logic for identifying the samples is defined as follows:

- **Positive Samples:** a temporal index i_t is selected if:
  - T_o contains no active link blockages (no `"1"` entries);
  - the first $T_p - 1$ steps of the prediction window also contain no blockages;
  - the exact *final* step of the prediction window is labeled as `"1"` (capturing the precise time instant of a connection drop, i.e. the exact beginning of a blockage).
- **Negative Samples:** a temporal index i_t is selected if:
  - the step immediately preceding T_o was labeled as `"1"` (meaning a previous blockage event has just concluded);
  - no blockages appear within the entire T_o;
  - no blockages appear also within the 16 time steps after T_o (meaning that the LOS link is unblocked for a time long enough to avoid having portions of pre-blockage signatures of further blockages inside negative samples).

By leveraging such sample selection rules, we enforce both positive and negative samples to be "perfectly clean", meaning that in positive samples we have no trace of *other* blockages and related consequences on receive power (apart from the one linked to that positive sample) and in negative samples we have no trace of *any* blockage and related consequences on receive power. Once identified, the extracted temporal indices i_t for each scenario are logged and saved to disk in dedicated text files (`scenarioX_pos_indices.txt` and `scenarioX_neg_indices.txt`), granting future reproducibility.

3.3.3 Sample Creation Phase

In the final phase, the script reads the pre-identified temporal indeces i_t to slice the corresponding 2D power heatmaps from the pre-cached multi-beam mmWave arrays. Crucially, to strictly align with the preprocessing methodology established by *Wu et al.* [5], a spatial cropping operation is applied to the extracted sequences: the 10 central beams, which correspond to the direct LOS path, are explicitly removed from the 64-beam codebook, since they have high power and this affects the detection of the pre-blockage signature in the signal. Consequently, each extracted input sample is shaped as a 16×54 two-dimensional matrix.

Since the sample identification phase gave as result different quantities of positive samples and negative samples, to prevent class imbalance from biasing the learning objective, an automated pruning routine forces class parity by randomly discarding entries from the majority class, in order to exactly match the total sample count of the minority class.
Following the balancing stage, the comprehensive array of positive and negative matrices is synchronized with their categorical labels, completely shuffled, and distributed according to a strict three-way dataset splitting ratio (`60%` training, `20%` validation, and `20%` testing). Finally, these subsets are exported to permanent storage in compressed NumPy archive format (`.npz`) under structured `train`, `val`, and `test` sub-directories assigned to each distinct T_p target variable.

3.4 Baseline Architecture

To establish a rigorous benchmark for our subsequent photonic adaptations, the initial phase of the modeling process involves replicating the CNN architecture proposed by *Wu et al.* [5]. This baseline model operates on the extracted input heatmaps, which have a shape of $16 \times 54 \times 1$.
The baseline architecture is structured as a sequential sequence of two main convolutional stacks followed by a dense prediction component.
The first stack consists of two `Conv2D` layers with 4 filters each and a 3×3 spatial kernel, using `ReLU` activations and zero-padding (`padding='same'`), terminating with a 2×3 `MaxPooling2D` downsampling operation.
The second stack deepens the feature extraction through a `Conv2D` layer with 8 filters and another with 16 filters, both still employing 3×3 kernels, followed again by 2×3 `MaxPooling2D`.
The high-level spatial features are then flattened into a 384-dimensional vector with a `Flatten` layer and regularized using a `Dropout` layer with a rate of 0.2. The final prediction component is a `Dense` layer with 2 units and a `Softmax` activation, which outputs the binary probability distribution.
To mitigate overfitting during the extended training runs, an *L2 Regularization* penalty ($\lambda = 10^{-3}$) is systematically applied to the weights of all convolutional and dense layers[2].

3.5 Photonic-Constrained Architecture

Transitioning from standard digital hardware to analog photonic accelerators necessitates the integration of strict structural physical constraints into the neural network topology.

[2] The introduction of *L2 Regularization* is the only structural difference between our baseline CNN and the one proposed by *Wu et al.* [5].

The first critical limitation of optical intensity modulation is the impossibility of representing negative data values. To strictly enforce this positivity constraint, our pipeline intercepts the dataset post-normalization: if the global minimum value of the training set is less than zero, an absolute translation offset is computed on the train set and universally added to the train, validation, and test matrices, shifting the entire data distribution strictly into the positive domain.

The second major structural constraint regards spatial downsampling. Standard non-linear pooling operations, such as *Max Pooling*, are computationally intractable for passive optical combiners. Consequently, all `MaxPooling2D` layers are replaced with `AveragePooling2D` operations, which compute linear spatial aggregations natively supported by photonics[3].

The third major structural constraint regards the width of the `Flatten` layer and comes as some sort of a consequence of the previous modification: since 384 is a too large amount of neurons for a single fully-connected layer [2], by leveraging the introduction of *Average Pooling*, we managed to halve it, taking it down to 196 neurons.

3.6 Quantization-Aware Training Pipeline

Because analog photonic engines suffer from noise and distortions that severely limit their Effective Number of Bits (ENOB), the full-precision constrained architecture must be quantized. To achieve this without catastrophic information loss, we deploy a Quantization-Aware Training (QAT) framework leveraging the Larq library. The architectural transformations include:

- **Input quantization:** the translated input data is quantized using a custom `fake_quant_with_min_max_args` operator, explicitly bounding the input dynamic range between 0.0 and 24.0[4].
- **Weight and activation quantization:** the continuous parameters of `Conv2D` and `Dense` layers are replaced by their Larq equivalents (`QuantConv2D` and `QuantDense`). We utilize the DoReFa quantization scheme, which maps weights to a $[-1, 1]$ uniform distribution using the hyperbolic tangent function, and clips activations between $[0, 1]$.
- **Normalization Strategy:** Because quantization heavily disrupts internal variance, `BatchNormalization` [4] is explicitly inserted after every quantized convolutional block and before the activation functions.

[3] *AveragePooling2D* layers are introduced after each single `Conv2D` layer (instead of only at the end of an entire convolutional stack) to limit the number of neurons in hidden layers of the network.

[4] After normalization and translation steps, all numerical values are inside the range of real numbers $[0.0, 24.0]$.

- **Activation Bounding:** Standard unbounded ReLUs are replaced by bounded ReLUs (`max_value=1.0`) to align with the strictly normalized quantization intervals.

This rigorous QAT methodology is systematically applied and evaluated across decreasing bit-width resolutions, specifically testing 16-bit, 8-bit, 4-bit and 2-bit quantization limits.

3.7 Architectural Explorations for Low Precision

As the simulated hardware resolution drops to 4 and 2 bits, the model faces severe representational bottlenecks. To investigate whether architectural modifications can compensate for extreme quantization noise, the 4-bit QAT methodology branches into three alternative topological evaluations:

1. **Deep Model variant:** explores depth scaling by introducing a third convolutional stack comprising two additional `QuantConv2D` layers (32 filters each) followed by average pooling. This expansion attempts to increase hierarchical feature extraction before the severe 4-bit and 2-bit density bottlenecks.

2. **Large Model variant:** explores width scaling by doubling the feature maps in the existing layers (i.e., expanding Stack 1 filters from 4 to 8, and Stack 2 filters from 8 to 16 and from 16 to 32).

3. **Small Model variant:** conversely tests the network's resilience by halving the filter sizes across all layers (i.e., reducing Stack 1 filters from 4 to 2, and Stack 2 filters from 8 to 4 and from 16 to 8). The motivation behind this structural reduction is to investigate whether a large parameter space, combined with the severe representational limitations of 4-bit quantization, induces overfitting or convergence instabilities. By restricting the total number of trainable parameters, this variant evaluates whether a more compact network can leverage its limited numerical precision more efficiently, preventing the dispersion of informational capacity across redundant weights.

These comprehensive topological explorations (*Deep*, *Large*, and *Small* variants) were systematically executed and evaluated exclusively within the 4-bit quantization domain. Consequently, for the ultimate and most aggressive hardware constraint (the 2-bit quantization execution) no further architectural adjustments were isolated. Instead, under the assumption that structural performance trends inherently mirror across ultra-low precision boundaries, the 2-bit framework directly inherited and evaluated only the top-performing architectural variant derived from the 4-bit analysis.

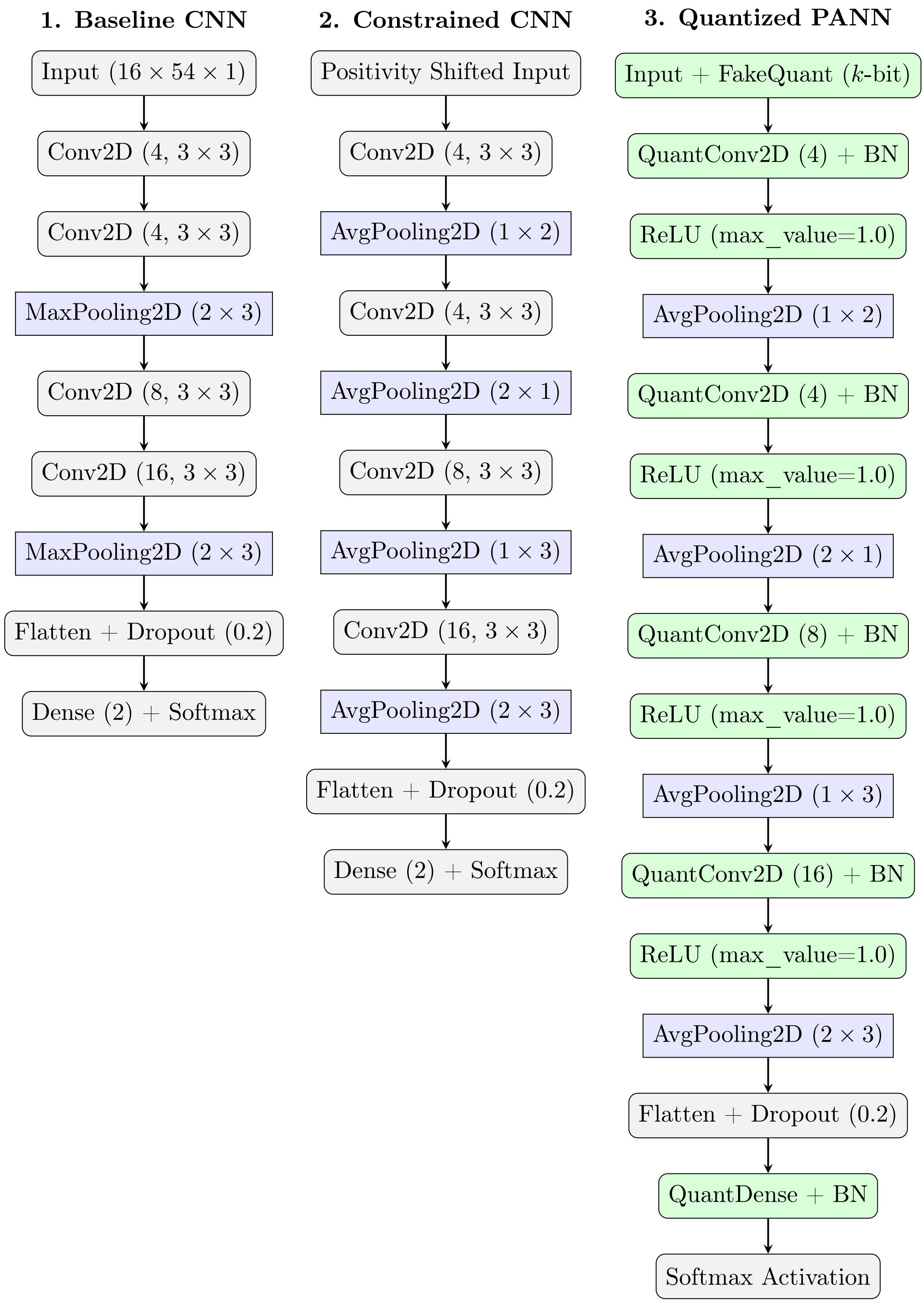


Figure 3.2: Incremental structural evolution of the network topology: (1) original full-precision baseline, (2) insertion of physical photonic constraints, and (3) final low-precision hardware configuration optimized via Larq Quantization-Aware Training.

3.7.1 Dataset Ingestion and Normalization Strategy

To stabilize gradient flows and accelerate network convergence during optimization, a *Z-score normalization* routine is integrated into the input data preparation layer [5]. The mean (μ_{train}) and standard deviation (σ_{train}) are computed exclusively on the training set to prevent data leakage. Every input feature heatmap $\mathbf{X}$ across all three partitions is then standard-scaled according to the relationship:

$$\mathbf{X}_{\text{norm}} = \frac{\mathbf{X} - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{3.2}$$

However, an empirical analysis of the data distributions following the standard Z-score normalization revealed a severe structural imbalance. While the normalization values spanned an overall theoretical interval of $[0.0, 24.0]$, the vast majority of the data points were densely concentrated within a narrow sub-range of $[0.0, 5.0]$. This pronounced skewness poses a critical bottleneck for the quantized models: such models heavily under-utilize the available discrete representation levels[5], concentrating the numerical values of the vast majority of the samples in the initial section of the representability interval. The immediate consequences are significant quantization errors and loss of feature resolution.

To address this limitation and fully exploit the representational capacity of the quantized models, we replaced the static Z-score formulation with a dynamic, non-linear transformation and binning pipeline parametrized by the target quantization bit-width (k). The primary objective is to map the skewed raw feature space into a uniform discrete space of 2^k distinct levels (ranging from 0 to $2^k - 1$). Given k as the target quantization bits, we define the number of possible discrete states S and the symmetry offset δ required for signal bounding as:

$$S = 2^k - 1, \quad \delta = \frac{S}{2}. \tag{3.3}$$

The revised normalization strategy operates in three sequential phases to enforce these constraints across all dataset partitions:

1. **Gaussian Mapping:** A `QuantileTransformer` is fitted exclusively on the flattened training partition to map the skewed distribution into a standard normal distribution. This mitigates the original skewness and evenly distributes the feature density.

[5] For instance, in the case of the 16-bit quantization, the representation interval of the quantizer (and, in general, of the entire model) is $[0, 2^{16} - 1] = [0, 65535]$, meaning that the vast majority of the real values are mapped to a restricted section of the integer values. The issue becomes significant from the 4-bit quantization onward, since the integer values range $[0, 2^4 - 1] = [0, 15]$ is narrower than the real values range $[0.0, 24.0]$, i.e. many real (different) values are collapsed to the same integer value, leading to a major loss in information.

2. **Symmetric Bounding:** A `MinMaxScaler` subsequently projects the Gaussian-distributed data into a symmetric continuous interval of $[-\delta, \delta]$.

3. **Input Translation and Quantization:** Finally, the continuous bounded tensor $\mathbf{X}_{\text{bounded}}$ is shifted by the symmetric offset $+\delta$, rounded to the nearest integer, and clipped to strictly enforce the k-bit hardware constraints. The mathematical operation applied to all dataset partitions is defined as:

$$\mathbf{X}_{k\text{-bit}} = \text{clip}\left(\lfloor \mathbf{X}_{\text{bounded}} + \delta \rceil, 0, S\right). \tag{3.4}$$

By forcing the data into this dynamically adapted discrete domain, the network learns to process features exactly as they will be represented in the physical photonic inference engine, while dynamically casting the tensors to the most memory-efficient format based on the value of k.

The processed arrays are subsequently encapsulated into high-performance data pipelines using the `tf.data.Dataset` API. To prevent localized gradient bias, the training partition is shuffled at the start of each epoch with a buffer size exactly matching the number of available sub-sampled entries.

All three subsets are partitioned into a fixed batch size of 64 samples.

3.7.2 Model Initialization and Optimization Parameters

From now on until the end of the paragraph 3.7.6, everything is to be strictly intended only on the *baseline* model architecture. Further explanations about *constrained* and *quantized* architectures are given in paragraph 3.7.7.

In order to guarantee a mathematically strict and unbiased comparative baseline across different configurations, the layer weights are reset and initialized deterministically using an explicit random seed parameter. The structural kernel weights of the convolutional and dense layers are populated using the He Normal distribution, while the corresponding bias terms are initialized uniformly to zero.

The optimization objective is formulated using a standard *Cross Entropy* loss function (using Python class `tensorflow.keras.losses.SparseCategoricalCrossentropy`), in accordance with *Wu et al.* [5].

The network parameters are optimized using *Stochastic Gradient Descent (SGD)* [3] (using Python class `tensorflow.keras.optimizers.SGD`): *Wu et al.* [5] does not explicit the optimizer used for training the models, so we opted for the simplest and most known one.

The *learning rate* used by SGD is tested across different orders of magnitude, ranging from 10^{-1} down to 10^{-6}: this allows a comprehensive assessment of optimizer convergence behavior across different step sizes. The learning rate resulting top-level from this test (10^{-3}) is then set and kept fixed for the training of all our models.

3.7.3 Training Dynamics and Callbacks

The network is permitted to train for a maximum ceiling of 2000 epochs, in accordance with *Wu et al.* [5][6]. To prevent *overfitting*, the internal execution loop is governed by a series of automated tracking callbacks:

- **Early Stopping mechanism:** to halt training once the generalization capacity limits are reached, an `EarlyStopping` callback actively tracks the validation loss curve. If no numerical improvement is observed over a window of 20 epochs, the entire execution loop is truncated.
- **Model checkpointing:** integrated directly alongside early stopping, a checkpointing callback monitors the validation loss at the end of each epoch and writes the network state to disk only when an absolute historical minimum is reached, ensuring that the final evaluated parameters correspond to the best-performing model state.

3.7.4 Performance Evaluation

Once training concludes, some training and validation metrics are gathered and plotted:

- *Training Loss* and *Validation Loss* are plotted one against the other to evaluate the presence of *underfitting* or *overfitting* phenomenons;
- *Training Accuracy* and *Validation Accuracy* are plotted one against the other to evaluate the general trend of our main metric (which is indeed *accuracy*).

Then, the baseline model is evaluated against the unseen test dataset to extract generalization metrics. Beyond recording the *Testing Accuracy*, in accordance with *Wu et al.* [5], the prediction pipeline extracts the raw output probability vectors across all evaluated test samples, which are compiled directly against the *ground-truth* labels to plot two analytical assessment structures:

- *Confusion Matrices (CM)* to visually dissect classification error distributions across the positive and negative link states;
- *Receiver Operating Characteristic (ROC) curves* to measure the predictive true-positive rate against false-positive rate.

[6]However, in the original paper there is no explanatory statement about any experimental justification to such (big) number of epochs. By looking at the training/validation loss plots of the very first models we trained, clear indications of overfitting emerge.

3.7.5 Multi-Seed Validation

In order to mitigate anomalies arising from specific random initializations and get to more statistically robust training, every individual architecture configuration is trained and tested across an array of independent five random initialization seeds ($\mathcal{S} = [1, 42, 1234, 1809, 2602]$[7]).
The training, validation and test metrics are gathered and saved separately for each of the 5 seeds. Also graphs regarding such metrics are plotted separately for each of them[8].
The final performance profiles are generated by computing the *arithmetic mean* of the test accuracies achieved across all seed iterations.

3.7.6 Experimental Paradigms

To analyze the behavior of the baseline architecture in an exhaustive and precise manner, the methodology organizes the baseline training routines into three sequential and distinct experimental paradigms. Each of them evaluates the full range of prediction windows ($T_p \in \{1, 2, \ldots, 10\}$).
The three execution paradigms are structured as follows:

1. **Standard baseline execution:** the first loop trains the unregularized network for a fixed sequence of 2000 epochs without early stopping. It serves as the primary benchmark to observe raw convergence behavior and baseline performance across all ten prediction windows.

2. **Early stopping execution:** the second loop introduces the validation-driven tracking mechanism. By setting the `early_stopping` flag to true, the training loop monitors generalization loss and saves the optimal weight configurations dynamically when validation loss reaches a historical minimum, preventing the network from overfitting during prolonged training cycles.

3. **Regularized and early-stopped execution:** the final, most robust execution framework layers both programmatic and structural constraints simultaneously. With both the `early_stopping` and `regularization` flags enabled, the pipeline instantiates the regularized model architecture,

[7]All seeds are chosen without any kind of ratio or logical relationship among them.

[8]In the present report, we include only the comparative results and visualizations evaluating multiple models or different iterations of a single model. Since the individual metrics detailed in Section 3.7.4 (specifically, loss and accuracy across epochs, confusion matrices, and ROC curves) are seed-specific, these results have been omitted to maintain the conciseness of the document. However, all the seed-specific materials produced throughout the case study can be easily accessed on the GitHub repository linked at the beginning of this report.

enforcing an L_2 weight degradation penalty ($\lambda = 10^{-3}$) inside the optimization objective while relying on validation checkpoints to isolate the optimal generalization limits.

3.7.7 Further Architecture Developments

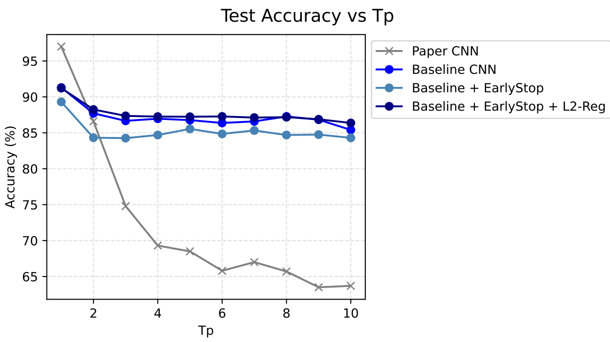
The analysis of the results on the three configurations of the baseline model pointed out that the best-performing training paradigm is the third one, i.e. including both *Early Stopping* and *L2 Regularization*. Therefore, we use this configuration as final *baseline* model and we prosecute the development of our case study by applying to it photonic constraints and quantization.

4. Experimental Setup

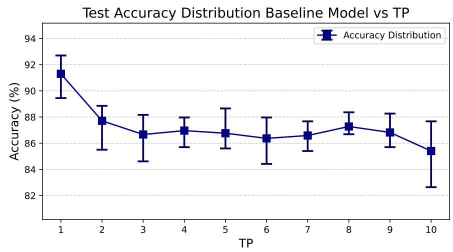
In this section we present the quantitative outcomes obtained across the different architectural model variants, including the baseline model, the photonic-constrained model and the quantized models. The implications and theoretical possible interpretations of these findings are subsequently addressed in the next chapter.

4.1 Evaluation of the Baseline Model

This section presents the outcomes for the *baseline* model across all ten prediction windows ($T_p \in \{1, \dots, 10\}$). To ensure a clear comparison, the performances of the three training paradigms of the baseline model are presented together in next graph, also compared with the reference paper curve. The cumulative trend of the arithmetic mean of test accuracies is instead presented only for the final baseline model (the one equipped with both L2-reg and Early Stopping).



(a) Comparative performance of *baseline* models vs. *Wu et al.* paper model.

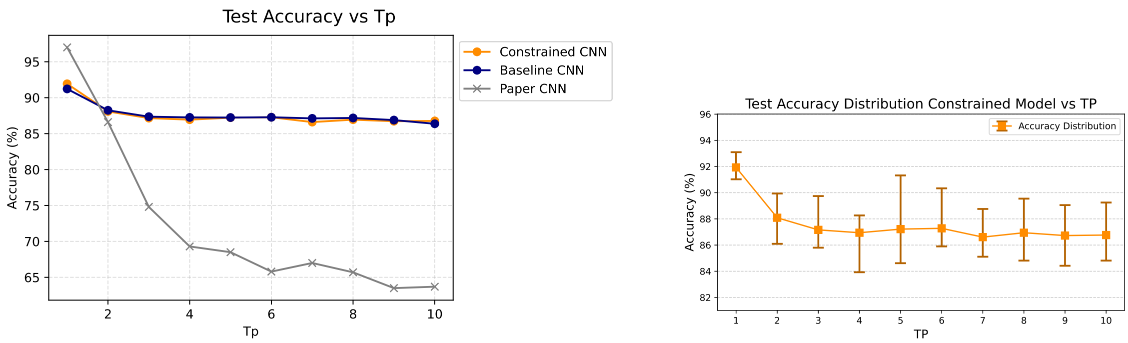


(b) *Baseline* model variance and error bounds.

Figure 4.1: *Baseline* CNN test accuracy performance profiles evaluated against changing T_p under different optimization and regularization paradigms.

4.2 Evaluation of the Photonic-Constrained Model

This section examines the performance of the neural network architecture when subjected to hardware photonic constraints, prior to the introduction of low-precision quantization. The objective of this experiment is to evaluate how these hardware-driven architectural boundaries impact the classification performance across the ten prediction windows ($T_p \in \{1, \ldots, 10\}$) compared to both our unconstrained baseline and the reference literature.



(a) Comparative performance of *constrained* model vs. previous models.

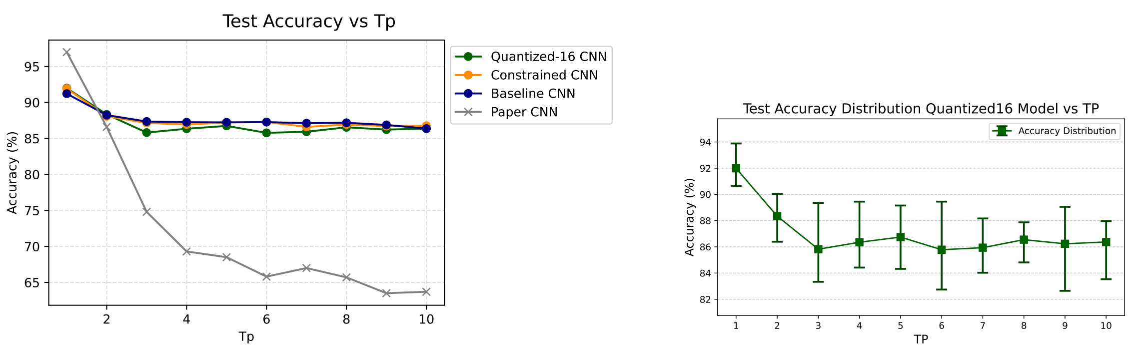
(b) *Constrained* model variance and error bounds.

Figure 4.2: *Constrained* CNN test accuracy performance profiles evaluated against changing T_p.

4.3 Evaluation of the Quantized Models

4.3.1 16-bit Quantized Model

This subsection focuses on the evaluation of the Quantization-Aware Training (QAT) pipeline configured at a 16-bit resolution for both network weights and activation functions. By employing simulated quantization nodes during the optimization phase, this step serves as the initial assessment of the network resilience against finite precision constraints. The primary objective is to determine whether a high-bit-width fixed-point representation can retain the predictive accuracy of the continuous parameters while adapting to the baseline noise typical of high-resolution analog accelerators.



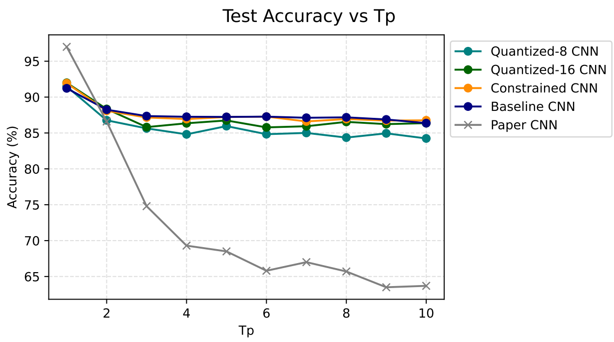
(a) Comparative performance of *quantized-16* model vs. previous models.

(b) *Quantized-16* model variance and error bounds.

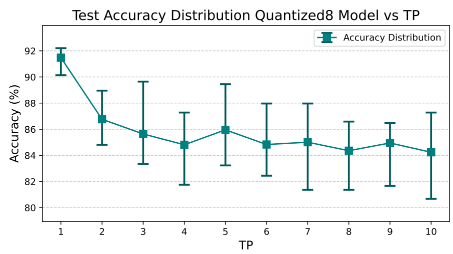
Figure 4.3: *16-bit quantized* CNN test accuracy performance profiles evaluated against changing T_p.

4.3.2 8-bit Quantized Model

This subsection presents the validation performance of the neural network architecture under an 8-bit quantization constraint. Moving to an 8-bit precision level significantly tests the capacity of the model to absorb truncation noise without degrading features: the focus is to determine if a narrower numerical scale can keep high results without requiring topological expansions.



(a) Comparative performance of *quantized-8* model vs. previous models.

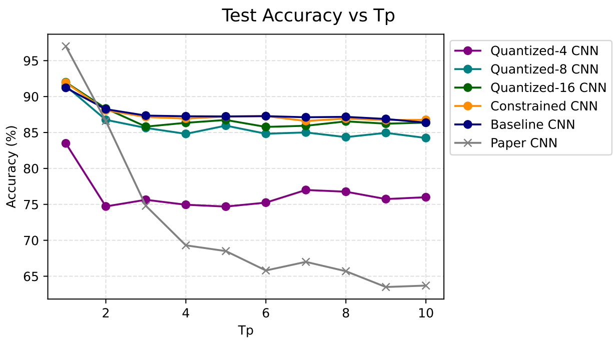


(b) *Quantized-8* model variance and error bounds.

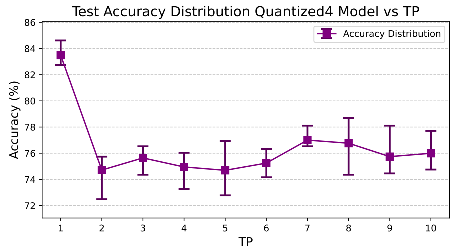
Figure 4.4: *8-bit quantized* CNN test accuracy performance profiles evaluated against changing T_p.

4.3.3 4-bit Quantized Model

This subsection provides the outcomes regarding the network performance under a 4-bit quantization layer, which reduces the numerical resolution of weights and activations to 16 discrete levels. Reducing the parameters to fewer discrete states restricts the network capacity, causing a drop in accuracy compared to high-precision models.



(a) Comparative performance of *quantized-4-basic* model vs. previous models.



(b) *Quantized-4-basic* model variance and error bounds.

Figure 4.5: *4-bit quantized* CNN test accuracy performance profiles evaluated against changing T_p.

The graphs just above are referred to the *basic* version of the *quantized-4* model, since it is the version that has been used for further architecture developments. Instead, in the following graph is reported the test accuracy performance comparison among all versions of the *quantized-4* model.

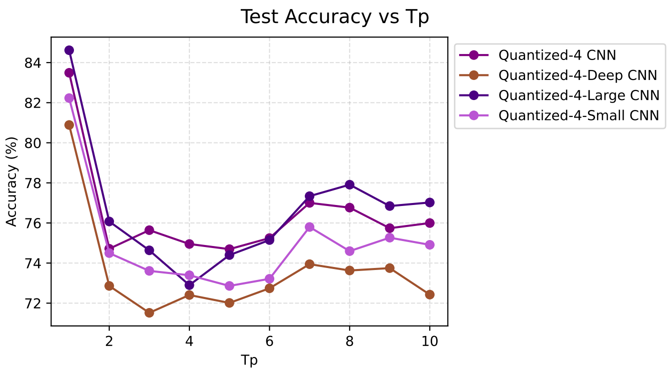


Figure 4.6: Comparative performance of the test accuracy of all *4-bit quantized* models.

4.3.3.1 Mitigation of 4-bit Quantization Error via Advanced Normalization

To address the performance drop caused by standard Z-score normalization, we reshaped the skewed input data into a uniform discrete space spanning exactly $[0, 15]$. This new strategy maximizes informational entropy across the available hardware states. We analyzed the feature density distribution across all dataset partitions (training, validation, and test splits): as illustrated in Figure 4.7, the features are strictly bounded and homogeneously distributed across the 16 available discrete states, effectively eliminating the right-skewness bottleneck observed in the baseline Z-score approach.

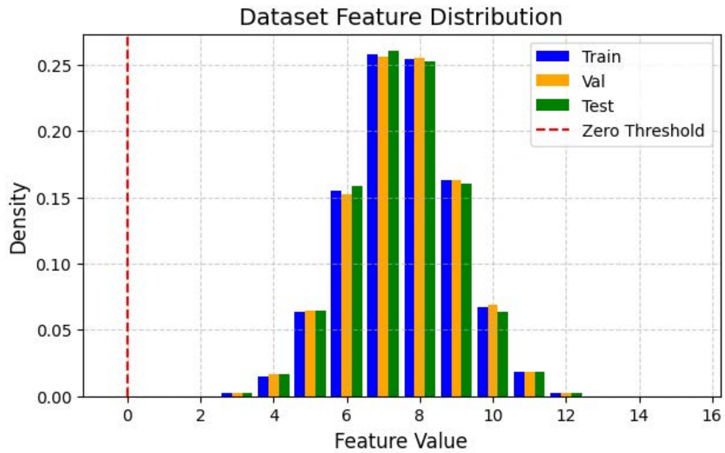


Figure 4.7: Empirical feature density distribution across the training, validation, and test partitions after applying the 4-bit dynamic normalization pipeline.

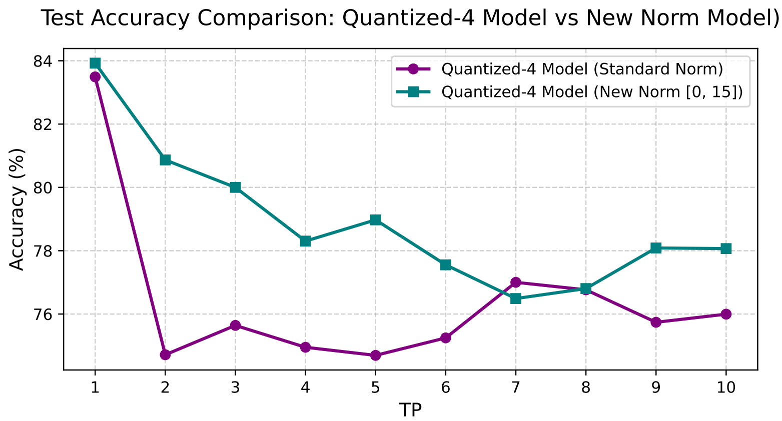
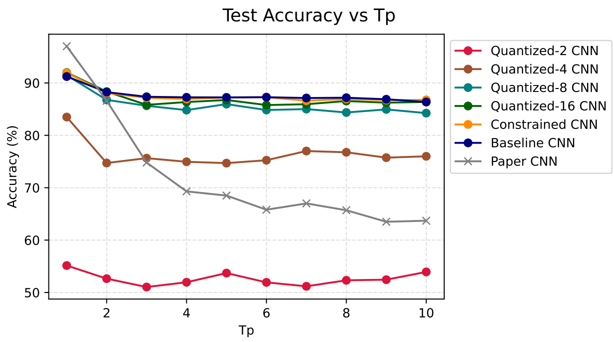


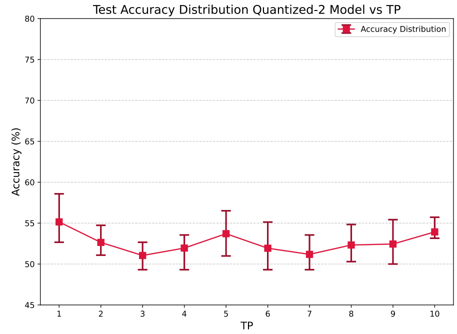
Figure 4.8: Comparative analysis of the *4-bit quantized basic* model under standard normalization vs. advanced Gaussian discrete mapping pipeline.

4.3.4 2-bit Quantized Model

This subsection evaluates the extreme constraints of a 2-bit quantization pipeline, where representational capability is strictly limited to merely 4 discrete states ($[0, 3]$) for both weights and activations. This configuration represents the most aggressive compression scenario. As expected, forcing the spatio-temporal features through such a narrow numerical bottleneck yielded a severe degradation in test accuracy when compared to all previous models.



(a) Comparative performance of *quantized-2* model vs. previous models.



(b) *Quantized-2* model variance and error bounds.

Figure 4.9: *2-bit quantized* CNN test accuracy performance profiles evaluated against changing T_p.

4.3.4.1 Mitigation of 2-bit Quantization Error via Advanced Normalization

The critical accuracy drop observed in the standard *2-bit quantized* model can be directly attributed to the mismatch between the original data distribution and the heavily restricted discrete space. Under standard Z-score normalization, the vast majority of the normalized (different) values were clustered into the same discrete value, leading to truncation when squeezed into just 4 states.

To counter this, we applied the dynamic k-bit parametrized non-linear transformation pipeline previously detailed in paragraph 3.7.1 and already used for the *4-bit quantized* model in paragraph 4.3.3.1. For the specific case of $k = 2$, the skewed raw feature are mapped and translated into the exact hardware interval of $[0, 3]$.

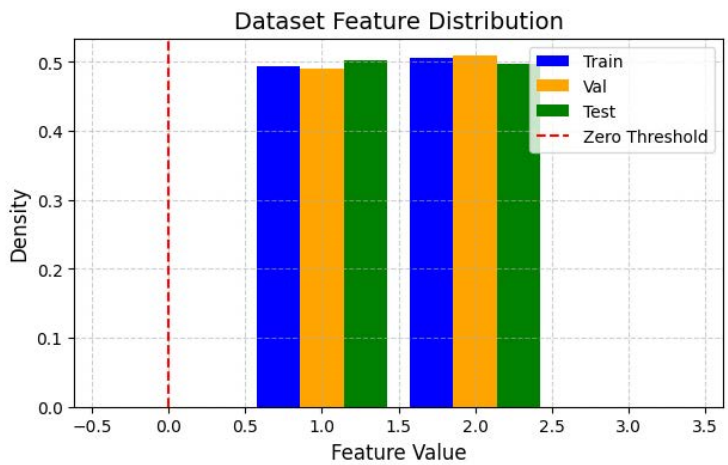


Figure 4.10: Empirical feature density distribution across training, validation, and test partitions after applying the 2-bit dynamic normalization pipeline.

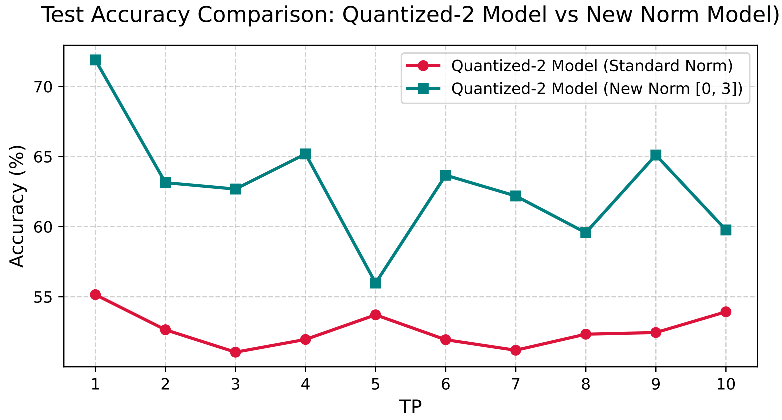


Figure 4.11: Comparative analysis of the *2-bit quantized basic* model under standard normalization vs. advanced Gaussian discrete mapping pipeline.

5. Discussion

The experimental data gathered across the different pipeline variants validate the architectural feasibility of deploying deep-learning proactive blockage prediction frameworks onto analog Photonic-Aware Neural Networks (PANNs).

5.1 Baseline CNN Performance

As shown in Figure 4.1a, our *baseline* CNN maintains a stable mean test accuracy tightly bounded between 86.5% and 87.4% for all extended prediction windows ($T_p \geq 2$).
In contrast, the reference model in literature [5] exhibits severe performance degradation, with its predictive accuracy dropping to 63.7% at $T_p = 10$. This architectural divergence implies that the underlying structural stability is primarily rooted in our algorithmic data preprocessing and sample identification pipeline. While the reference study [5] relies on *"manual"* [5] trajectory-based labeling (which introduces potential heuristic variance and constrains exact empirical replicability), our algorithmic pipeline for isolating valid positive and negative samples preserve structural feature consistency. Moreover, the combination of *L2 Regularization* and *Early Stopping* operates as a refinement mechanism that slightly expands the test accuracy boundary by an additional 1% or 2% depending on the specific T_p.

5.2 Impact of Photonic Hardware Constraints

The architectural conversion into a Photonic-Constrained framework introduces no statistically meaningful performance penalties, yielding an average accuracy divergence of less than 0.2% across all ten target variables compared to the unconstrained baseline, as shown in Figure 4.2a.
This parity yields two fundamental insights:

- The linear spatial aggregation (*Average Pooling*) inherently computed by optical combiners perfectly substitutes standard digital nonlinearities (*Max Pooling*) when extracting wave signatures from millimeter-wave power heatmaps.
- Enforcing input positivity via a uniform translation offset preserves the underlying structural topology of the signal fluctuations without scattering the discriminant information.

5.3 High-Resolution Quantization (16-bit & 8-bit)

As the quantization depth shifts toward lower bit-width spaces under the Quantization-Aware Training (QAT) paradigm, the network displays significant resilience at higher resolutions.
The *16-bit quantized* model achieves almost the same accuracy as the full-precision baseline, stabilizing at an average value of approximately 86% across the look-ahead range, as detailed in Figure 4.3a.
When compressing the parameter and activation spaces further to an 8-bit DoReFa mapping, the architecture successfully absorbs the progressive truncation noise, exhibiting only a minor performance attenuation (see Figure 4.4a) that holds a mean accuracy of 84.2% at the maximum prediction window ($T_p = 10$).

5.4 Low-Resolution Quantization (4-bit & 2-bit)

As the numerical precision drops below the 8-bit threshold into the *4-bit quantized* and *2-bit quantized* structural domains under a standard Z-score data transformation scheme, the network encounters a severe representational collapse.
As illustrated in Figure 4.5a, the *4-bit quantized* CNN experiences a substantial drop in test accuracy, losing approximately 10% compared to the *8-bit quantized* counterpart across all the prediction windows.
More critically, observing the chart shown in Figure 4.9a, the 2-bit model fails to show any learning progress, leaving the model performing no statistically better than a random uniform classifier (accuracy $\approx$ 50% across all the prediction windows).
Our topological architectural explorations within the 4-bit domain described in section 3.7 yield almost negligible performance improvements or even important degradation, as observed in Figure 4.6. This empirical behavior provides an essential insight: the dramatic accuracy loss observed in ultra-low precision regimes is not a consequence of an unoptimized parameter[1] space or a lack of representational depth within the parameterized feature maps. Instead, it is governed by an informational capacity bottleneck located directly at the data ingestion layer: the major improvement at these quantization levels are direct consequence of data processing techniques.

[1] Meaning the parameters of the model, such as amount of layers or amount of channels per layer.

5.4.1 Accuracy Recovery via Advanced Normalization

The introduction of the new normalization, described in section 3.7.1, acts as a definitive solution to this representational bottleneck by explicitly optimizing the information entropy across the available optical hardware states ($2^k - 1$).

5.4.1.1 4-bit Quantization Improvement

The quantitative impact of this entropy-balancing mechanism is most prominent within the optimized 4-bit domain: as shown in Figure 4.8, the advanced normalization strategy achieves an accuracy recovery, forcing the mean test performance back to 84% at short prediction windows ($T_p = 1$) and consistently preserving high predictive capabilities (bounded between 76.5% and 79%) even across the most extended forward prediction windows ($T_p \geq 4$).

5.4.1.2 2-bit Quantization Improvement

Within the ultra-low precision 2-bit regime, analyzing the comparison graph illustrated in Figure 4.11, the gap between standard and advanced normalization is evident.
With standard normalization, the 2-bit-quantized model shows extremely low and unstable performance, whilst the new normalization acts as a stabilizer, allowing the model to maintain higher accuracy in the case of almost all the prediction windows (oscillating between 60% and 72%). The only exception is $T_p = 5$, which results in an accuracy of about 56%, but still remaining slightly higher than the standard normalization variant.
All these findings demonstrate that the entropy balancing mechanism is crucial when the bit budget is so limited, exactly as we had previously assumed.

5.5 Final conclusions

Our work has demonstrated the concrete viability of Photonic-Aware Neural Networks (PANNs) for the proactive management of link blockages in mmWave scenarios. Through the integration of Quantization-Aware Training (QAT) techniques and an adaptive normalization strategy, we have mitigated the precision constraints imposed by optical hardware, ensuring robust performance even in low precision regimes.
These findings suggest that the physical limits of analog systems do not preclude the adoption of advanced artificial intelligence technologies in high-efficiency communication systems.

5.6 Future Work

While the present study demonstrates the architectural feasibility of PANNs for proactive blockage prediction, several directions remain open for further investigation.

- All photonic constraints and quantization effects were simulated in software through the QAT pipeline. A natural extension of this work involves validating the proposed architecture on physical photonic accelerators. Real world analog optical systems introduce noise sources (e.g. phase noise, thermal drift) beyond simple bit-width truncation. Evaluating the model's robustness against these physical phenomena is crucial, as well as directly quantifying the actual latency and energy consumption savings.
- Our study restricts its scope to the binary prediction subproblem introduced by *Wu et al.* [5]. Extending the photonic constrained and quantized methodology to the remaining subproblems investigated in the original work namely *blockage instance prediction* (timing), *severity classification* (type) and *moving direction estimation* would provide a more complete picture of the trade offs induced by analog hardware constraints across different task complexities.
- While the dynamic normalization strategy proved highly effective for input data, mitigating the residual quantization noise in the hidden layers remains an open challenge, especially at extreme 2 bit precision (e.g. at prediction window $T_p = 5$). Future efforts should focus on exploring more advanced QAT algorithms specifically designed to preserve feature resolution in the deeper stages of the network.
- Finally, extending the experimental evaluation to encompass the complete set of outdoor vehicular environments (Scenarios 17-22), as well as the indoor environment (Scenario 16) provided by the DeepSense 6G dataset, would critically assess the generalization capability of the proposed photonic constrained architecture under vastly different signal propagation and mobility conditions.

Bibliography

[1] Ahmed Alkhateeb, Gouranga Charan, Tawfik Osman, Andrew Hredzak, João Morais, Umut Demirhan, and Nikhil Srinivas. Deepsense 6g: A large-scale real-world multi-modal sensing and communication dataset. *arXiv preprint arXiv:2211.09769*, 2023. URL: https://deepsense6g.net/.

[2] Emilio Paolini, Lorenzo De Marinis, Marco Cococcioni, Luca Valcarenghi, Luca Maggiani, and Nicola Andriolli. Photonic-aware neural networks. *Neural Computing and Applications*, 2022. URL: https://doi.org/10.1007/s00521-022-07243-z.

[3] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv:1609.04747*, 2017. URL: https://doi.org/10.48550/arXiv.1609.04747.

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv:1502.03167*, 2015. URL: https://doi.org/10.48550/arXiv.1502.03167.

[5] Wu Shunyao, Alrabeiah Muhammad, Chakrabarti Chaitali, and Alkhateeb Ahmed. Blockage prediction using wireless signatures: Deep learning enables real-world demonstration. *IEEE Open Journal of the Communications Society*, 3, 2022. doi:10.1109/OJCOMS.2022.3162591.